AttentionLens

The Whole Pipeline — from image to verdict



Project Review 1 · B.Tech CSE (AI & ML), VIT

Jaswanth Krishna Padarthi

M1 delivered · M2–M8 roadmapped · 15 slides

Agenda

From a raw image to a bias verdict — the road map

01	**Problem & Motivation**	Why vision models need a debugger, not just explanations
02	**Design — Dual-View Schema**	One LayerMap contract, two views, CNN ⇄ ViT
03	**The Pipeline — 6 Stages**	Preprocess → Infer → Extract → Metrics → Intervene → Screen
04	**Stage-by-Stage Detail**	What each stage emits and what it costs
05	**Milestones M1–M8**	What's delivered, what's next, and why
06	**Validation Experiments**	Planted-bias (M7) + ObjectNet (M8) — both required
07	**Novelty & Honest Limits**	Tool-not-method · dual-view · auto-screening · known failures
08	**Next Steps**	M2 ViT extractor — the next gate

Problem & Motivation

Explanations exist — but they don't talk to each other

The field is fragmented

- Attention maps (ViT) answer 'what does the model look at' — but they are class-agnostic.
- Attribution maps (Grad-CAM) answer 'what drove this decision' — class-specific.
- No shared schema: comparing two architectures or two views today means bespoke glue.

Bias goes unnoticed

- Spurious correlations (backgrounds, textures, artifacts) silently inflate accuracy.
- Finding them today is manual, ad-hoc, and rarely automated end-to-end.
- Engineers need a verdict — not another heatmap to eyeball.

One framework, two jobs

① A unified CNN ⇄ ViT abstraction — every layer yields a standard LayerMap (native attention + class-conditioned relevance). ② An automated bias-screening layer — attention-quality metrics and intervention experiments that produce a verdict, in the tradition of SHAP / Captum as a deployable tool.

Design — the Dual-View LayerMap

Every layer, two views, one schema

Native view — what the model attends to

- Mechanism's own signal (CNN channel-mean, ViT attention rollout)
- Class-agnostic — broad by design, high entropy
- Answers 'where does the mechanism focus?'

Class-conditioned view — why the model decided

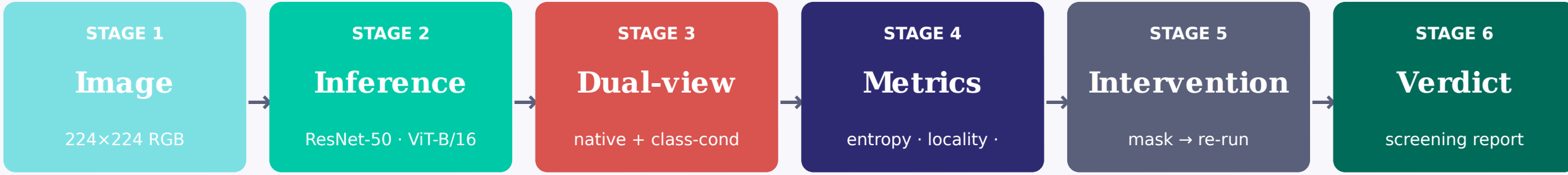
- Decision evidence (Captum LayerGradCam; class-weighted rollout / LRP / token attr for ViT)
- Class-specific — sharp, ReLU'd, evidence for one class
- Screening verdicts always rest on this view (the comparable one)

Why two views? Naive comparison is meaningless.

ViT attention and Grad-CAM are not the same kind of thing — the first is class-agnostic info flow, the second is class-specific decision evidence. Putting both behind one LayerMap dataclass makes CNN ✗ ViT comparison honest by construction. Verified by tests/test_schema.py on Arjun.

The Pipeline — six stages, one loop

Each stage emits; the loop closes on a verdict



Stage 5 (intervention) can re-enter Stage 3 with a masked image → evidence is causal, not correlative.

Stage 1	Tensor [1,3,224,224] fp32 (reversible to pixels)
Stage 2	(label, class_idx) — class for conditioning
Stage 3	LayerMap × (N layers × 2 views) — the unified contract
Stage 4	per-map metrics (object rel., entropy, part align., …)
Stage 5	InterventionResult (region, pred_before, pred_after)
Stage 6	Verdict + evidence links + report

01 Preprocessing

Stage 1 — Preprocessing

Resize → /255 → ImageNet normalize → tensor. Reversible for overlay.

load RGB → resize 224×224 → ÷255 → [0,1] → −mean ÷std → [1,3,224,224] fp32

Raw pixel stats (0–1)

R	G
0.388 ± 0.320	0.431 ± 0.295

Normalized tensor stats

R	G	B
-0.421 ± 1.398	-0.110 ± 1.316	-0.170 ± 1.498

- Design point: reversible — tensor_to_image() inverts every step exactly, so heatmaps overlay on real pixels.
- Stats above measured on Arjun — Python 3.12.10, torch 2.13.0+cpu.
- Tests: test_preprocess.py covers round-trip equivalence (hermetic — no downloads).

02 Inference

Stage 2 — Inference

One forward pass per model — yields the class that conditions every class-conditioned map.

ResNet-50

✅ **M1 — wired**

```
resnet50(weights=IMAGENET1K_V2)
```

Weights: ImageNet-1K V2 (76% top-1)

Role: CNN baseline

ViT-B/16

⏳ **M2 — planned**

```
timm.create_model('vit_base_patch16_224', pretrained=True)
```

Weights: ImageNet-1K

Role: Transformer comparison

Output contract: (label_str, class_idx) → becomes the conditioning target for every class-conditioned map downstream. Labels come from weights.meta["categories"] — a dict on torchvision 0.28 (gotcha: never getattr(meta, "categories")).

03 Dual-view extraction

Stage 3 — Dual-view extraction

Four probe points × two views — every probe emits a native and a class-conditioned LayerMap.

layer1.2.conv3	layer2.3.conv3	layer3.5.conv3	layer4.2.conv3
256 ch	512 ch	1024 ch	2048 ch
edges / textures	patterns / parts	object parts	object semantics

Native (mechanism)

- Forward hook on the layer → activations[B,C,H,W]
- mean over channels → bilinear upsample → normalize [0,1]
- Cost: one forward pass total (hooks capture all layers)
- Class-agnostic — broad by nature

Class-conditioned (evidence)

- Captum LayerGradCam on the same conv, target = predicted class
- ReLU → bilinear upsample → normalize [0,1]
- Cost: one attribution per layer
- Honest failure: all-zero map ⇒ 'no positive evidence at this layer'

Emits: 8 LayerMaps per image (4 blocks × 2 views) — the contract every downstream stage reads.

04 Metrics

Stage 4 — Metrics

Numbers, not vibes — measured against CUB-200 bboxes + part keypoints.

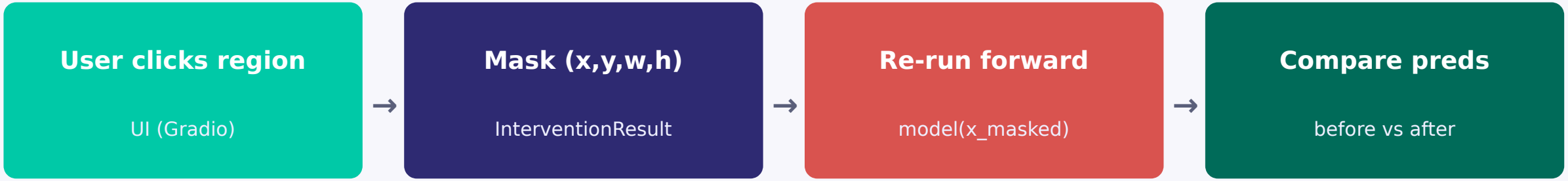
Object relevance share	fraction of attention mass inside the object's bbox	*Did the model look at the object or somewhere else?*
Part alignment	attention mass over annotated keypoints (beak, eye, wing)	*Did it find the right part of the object?*
Focus entropy	spread of the attention map	*Broad background-y vs sharp object-y*
Background / object sensitivity	prediction shift when the background is masked	*How much does it lean on the background?*

Why CUB-200? Bounding boxes + part keypoints — metrics measurable against annotations, not eyeballs. DECISIONS §4.

05 Intervention

Stage 5 — Intervention loop

Mask a region → re-run → pred_before ≠ pred_after is causal proof, not correlation.



InterventionResult — the evidence packet

- region (x, y, w, h) in input coordinates
- pred_before e.g. "Samoyed"
- pred_after e.g. "white wolf"
- logit_delta e.g. -4.21 (how much the prediction moved)
- metadata (mask type, baseline confidence, …)

M4 gate: masking the head must flip bird predictions on CUB-200. If it doesn't, the explanation was wrong — and the tool tells you so.

06 Screening

Stage 6 — Screening & verdict

Aggregate per-image metrics → class-level verdicts → evidence-backed report.

Batch inspections
(model, image) × N

→

Per-image metrics
4 metrics × 8 maps

→

Class-level aggregation
group by class

→

Verdicts + evidence links
bias flag · role profile · failure dx

Class bias flag	'wolf' classified on snow, not wolf
Layer-role profile	'layer 1 = edges, layer 4 = object'
Failure diagnosis	'misclassified because attention hit the bg'

Verdicts always rest on class-conditioned evidence — the comparable, trustworthy view (DECISIONS §2).

Milestones M1–M8

Each milestone ends with a verification gate

Milestone	Scope	Status	Verification gate
M1	Schema + ResNet-50 dual-view	✅ **done**	CUB image → 8 LayerMaps, sane maps
M2	ViT-B/16 dual-view (rollout + class-cond)	⏳ **next**	Same image → same schema, dual-view complete
M3	Metrics (object rel · part align · entropy · sens)	⏳	Numbers sane on known cases
M4	Intervention loop (mask → re-run)	⏳	Masking the head flips bird preds on CUB-200
M5	Gradio UI (per-layer grid + click-to-mask)	⏳	End-to-end interactive session
M6	Screening engine (aggregate → verdicts)	⏳	Flags fire on a known-biased model
M7	Planted-bias (Waterbirds-style) on CUB-200	⏳	Screener catches the exact planted shortcut
M8	ObjectNet natural-bias + wrap-up	⏳	Screener flags context reliance

M1 fully verified end-to-end (Samoyed, 8 maps, correct prediction, commit 96c50b6). M2 is the next gate.

Validation — two killer experiments

Proves the tool catches what we know + what's real

M7 · Planted-bias (Waterbirds-style)

- Setup: synthetic background/species correlation on CUB-200 → fine-tune ViT (+ResNet)
- What it tests: does the screener catch a shortcut we deliberately planted?
- Pass criterion: flag fires on the planted bias; bg/object sensitivity ≫ 1
- Why it matters: proves the tool catches what we KNOW is there

M8 · ObjectNet natural-bias

- Setup: standard pretrained ImageNet model on objects in bizarre contexts
- What it tests: does the screener catch context reliance we did NOT plant?
- Pass criterion: failure-diagnosis explains the misclassifications
- Why it matters: proves the tool catches what's REAL

Both required. A screening tool is only science if its verdicts are provably true positives — known bugs caught AND real bugs caught.

Novelty & honest limits

Engineering novelty beats another Grad-CAM variant

What is genuinely new

- 1. Unified dual-view abstraction — native + class-conditioned behind one schema, for both CNN and ViT. Apples-to-apples by construction.
- 2. Automated screening verdicts — class-level bias flags, layer-role profiles, evidence-backed reports. No other tool ships this.
- 3. Causal debugging loop — intervene → measure → verdict. Heatmaps are correlative; interventions are causal.
- Combination (1+2) is the novelty anchor — neither alone is rare; together they are.

What's not new (and that's fine)

- Tool, not method — we don't invent a new attribution algorithm. The XAI method space is saturated (Grad-CAM + 50 variants, attention rollout, LRP…).
- Engineering novelty (a genuinely useful tool) beats another Grad-CAM variant — this is how SHAP and Captum earned their place.
- Honest failure: Grad-CAM can yield all-zero maps (observed at layer2.3 on the Samoyed). Reported, not faked — DECISIONS §9.
- GPU/scale runs are code-ready (device-agnostic) but not yet exercised at scale. M2+ moves to rented GPU / Jenga.

Honest open items (logged, not hidden): zero-map degeneracy guard · ViT class-conditioning method pick (M2) · GPU-scale validation.

Thank you

Questions & discussion

Next gate — M2: ViT extractor. Same schema, dual-view, faithfulness check.

